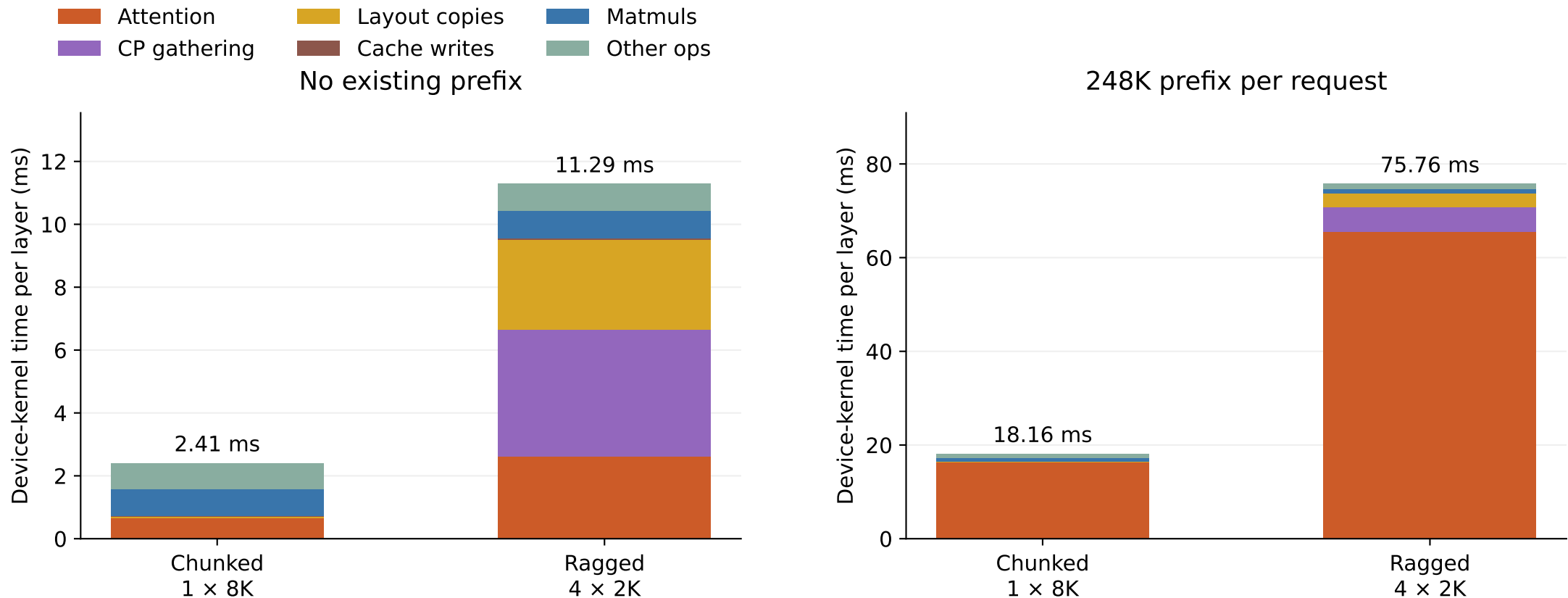


Global attention layers

One chunked 8K request vs one ragged 4×2K batch. Every bar processes 8,192 useful tokens.



Beginning: largest added costs are cp gathering +4.02 ms, layout copies +2.82 ms. 248K prefix: largest added costs are attention +49.07 ms, cp gathering +5.36 ms.

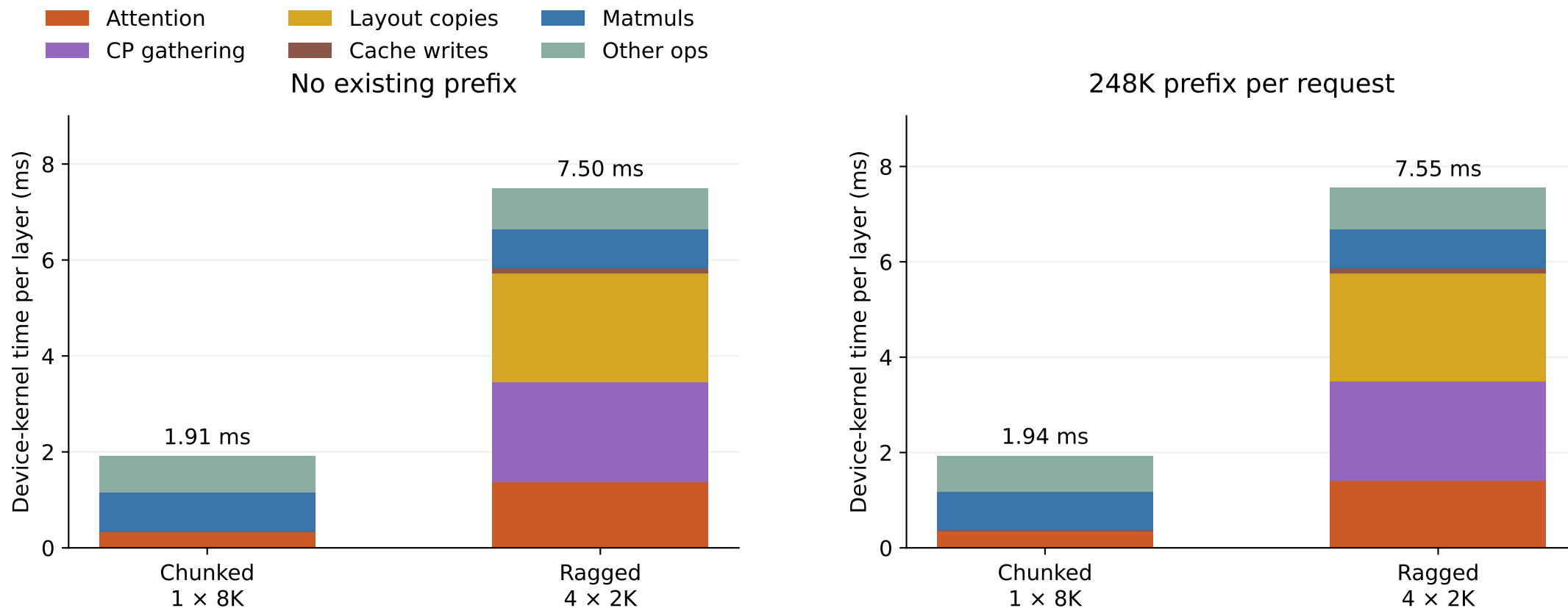
Per layer: attention calls 1 -> 4; extra CP gathers 6; matmuls 5 -> 5. Each 2K request is padded to 8K: four calls process 32K query rows for 8K useful rows. Panels use different y scales.

CP gathering collects token rows from the eight context-parallel devices. Layout copies slice, pad, repartition and reassemble those rows. Other ops include TP collectives, norms and RoPE.

Isolated layer, one traced replay per cell, CP8/TP4, four 256K cache slots initialized with random values. Embedding, RoPE preparation and host staging are outside the timed layer. Both paths use reduce-scatter. Bars sum device-kernel times: tt-perf-report 1.4.0 uses the maximum across devices for ordinary ops and the average for collectives. These are profiled per-layer measurements, not 60-layer model latency.

Sliding-window attention layers

One chunked 8K request vs one ragged 4×2K batch. Every bar processes 8,192 useful tokens.



Beginning: largest added costs are layout copies +2.26 ms, cp gathering +2.08 ms. 248K prefix: largest added costs are layout copies +2.25 ms, cp gathering +2.09 ms.

Per layer: attention calls 1 -> 4; extra CP gathers 7; matmuls 5 -> 5. Each 2K request is padded to 8K: four calls process 32K query rows for 8K useful rows. Panels use different y scales.

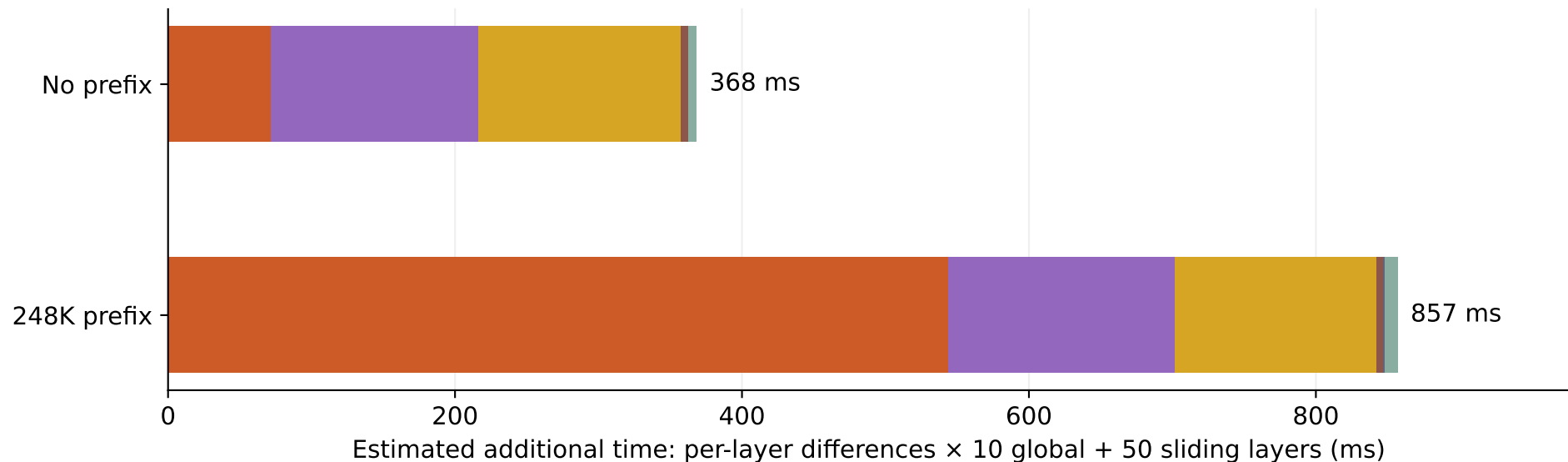
CP gathering collects token rows from the eight context-parallel devices. Layout copies slice, pad, repartition and reassemble those rows. Other ops include TP collectives, norms and RoPE.

Isolated layer, one traced replay per cell, CP8/TP4, four 256K cache slots initialized with random values. Embedding, RoPE preparation and host staging are outside the timed layer. Both paths use reduce-scatter. Bars sum device-kernel times: tt-perf-report 1.4.0 uses the maximum across devices for ordinary ops and the average for collectives. These are profiled per-layer measurements, not 60-layer model latency.

What explains the full-model gap?

One chunked 8K request vs one ragged 4×2K batch; both process 8,192 useful tokens.

Attention Layout copies Matmuls
CP gathering Cache writes Other ops



Earlier full-model measurements (chunked -> ragged, median of five replays): no prefix 130.4 -> 485.9 ms, an extra 355.5 ms; 248K prefix 312.7 -> 1154.9 ms, an extra 842.2 ms.

Why measure both layer types? There are 50 sliding layers and 10 global layers. Repeated data gathering and padding accumulate across all 60. Global attention performs four full-size passes over cached history, making its wasted padded-query work especially costly near 256K.

The bars are an extrapolation from isolated-layer kernel timings, not an exact breakdown of wall time. The earlier full-model measurements used populated histories; these profiles use random caches. The layer mix explains the size and direction of the gap, without implying every layer has identical timing. This is performance analysis, not numerical validation; the earlier report records an unresolved 60-layer correctness failure for a separate [8192, 33] batch.